IV.1. Organizarea calculatorului

Componentele principale

- Unitatea centrală de procesare (CPU)
- Memoria
- Dispozitive periferice (I/O = *input*/*output*)
- Magistrale
  - de date
  - de adrese
  - de control

Unitatea centrală de procesare

- numită şi procesor
- execută instrucţiunile indicate de programator
- realizează prelucrarea datelor
- coordonează funcţionarea celorlalte componente

Memoria

- stocarea informaţiilor
  - date
  - instrucţiuni
- furnizarea informaţiilor la cerere
- rol pasiv
  - "răspunde" la cererile venite din exterior
  - nu iniţiază niciodată un transfer

Dispozitivele periferice

- comunicarea cu exteriorul
- funcţii - foarte variate
  - preluare date
  - afişare
  - imprimare
  - stocare (persistentă)
  - etc.

Magistralele

- căi de legătură între CPU, memorie şi periferice
- după informaţia care le parcurge
  - de date - date şi instrucţiuni
  - de adrese - adrese pentru memorie şi dispozitive periferice
  - de control - semnale prin care CPU comunică cu celelalte circuite şi le controlează

IV.2. Memoria

Tipuri de memorie

- ROM (*Read-Only Memory*)
  - conţinutul său poate fi citit, dar nu şi modificat
  - nevolatilă (nu îşi pierde conţinutul la întreruperea alimentării)
- RAM (*Random Access Memory*)
  - conţinutul său poate fi citit şi modificat
  - volatilă (îşi pierde conţinutul la întreruperea alimentării)

Memoria ROM - tehnologii

- PROM (*Programmable ROM*) - conţinutul său poate fi programat de utilizator
- EPROM (*Erasable PROM*) - poate fi şters şi reprogramat de mai multe ori
  - UVEPROM (*Ultra-Violet EPROM*) - ştergere prin expunere la radiaţie UV
  - EEPROM (*Electrical EPROM*) - ştergere prin impulsuri electrice

Memoria RAM - tehnologii

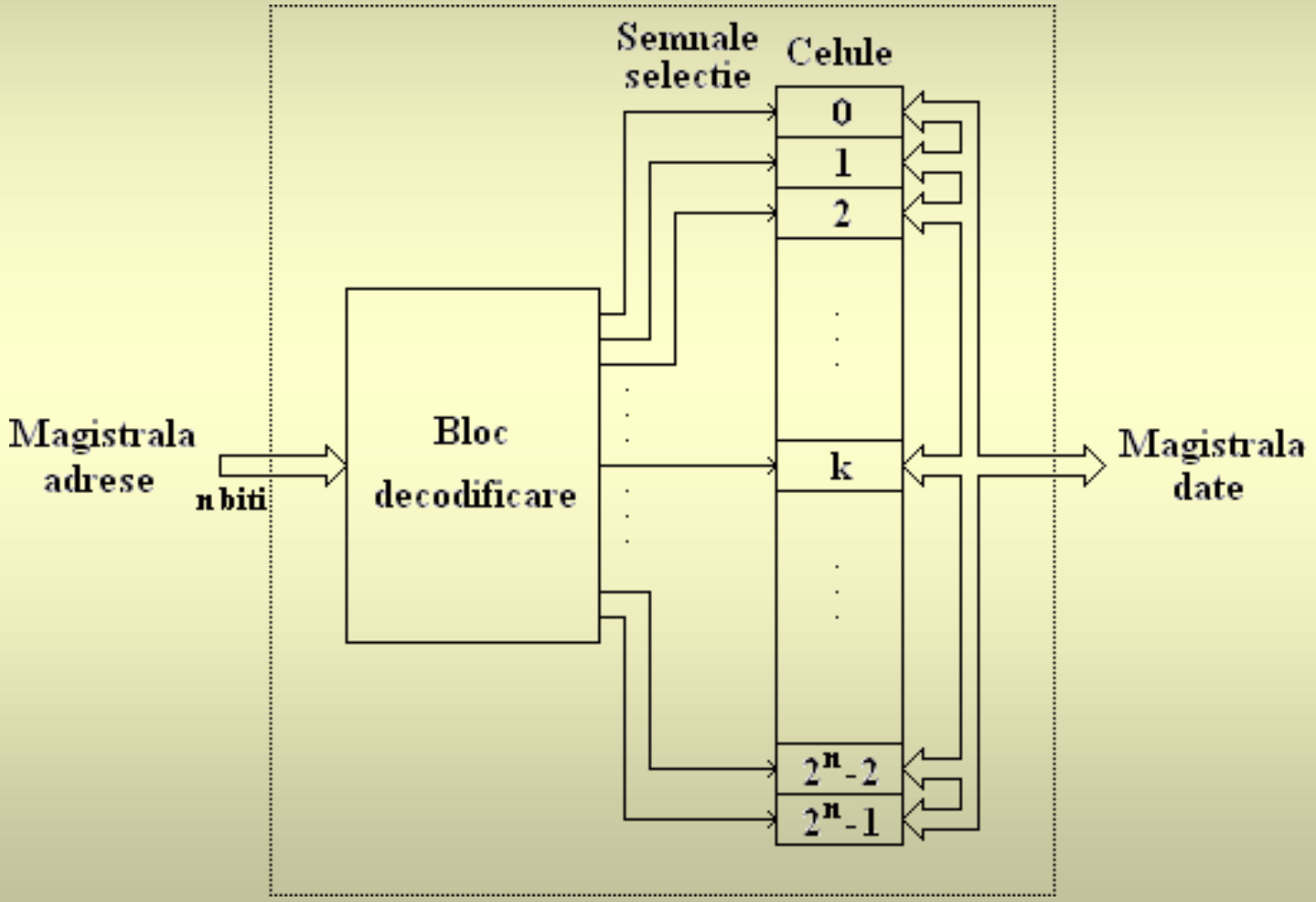
- SRAM (*Static RAM*)
  - viteză mare
  - preţ ridicat
- DRAM (*Dynamic RAM*)
  - mai lentă
  - densitate de integrare mare → spaţiu ocupat mic
  - preţ mai redus

Structura memoriei (1)

- şir unidimensional de celule (locaţii)
- fiecare celulă are asociat un număr unic - *adresa*
- bloc decodificare - selectează locaţia cu adresa indicată
- dimensiunea circuitului de memorie - dată de numărul de biţi de adresă

$$dimensiune = 2^{nr_biti_adresa}$$

Structura memoriei (2)



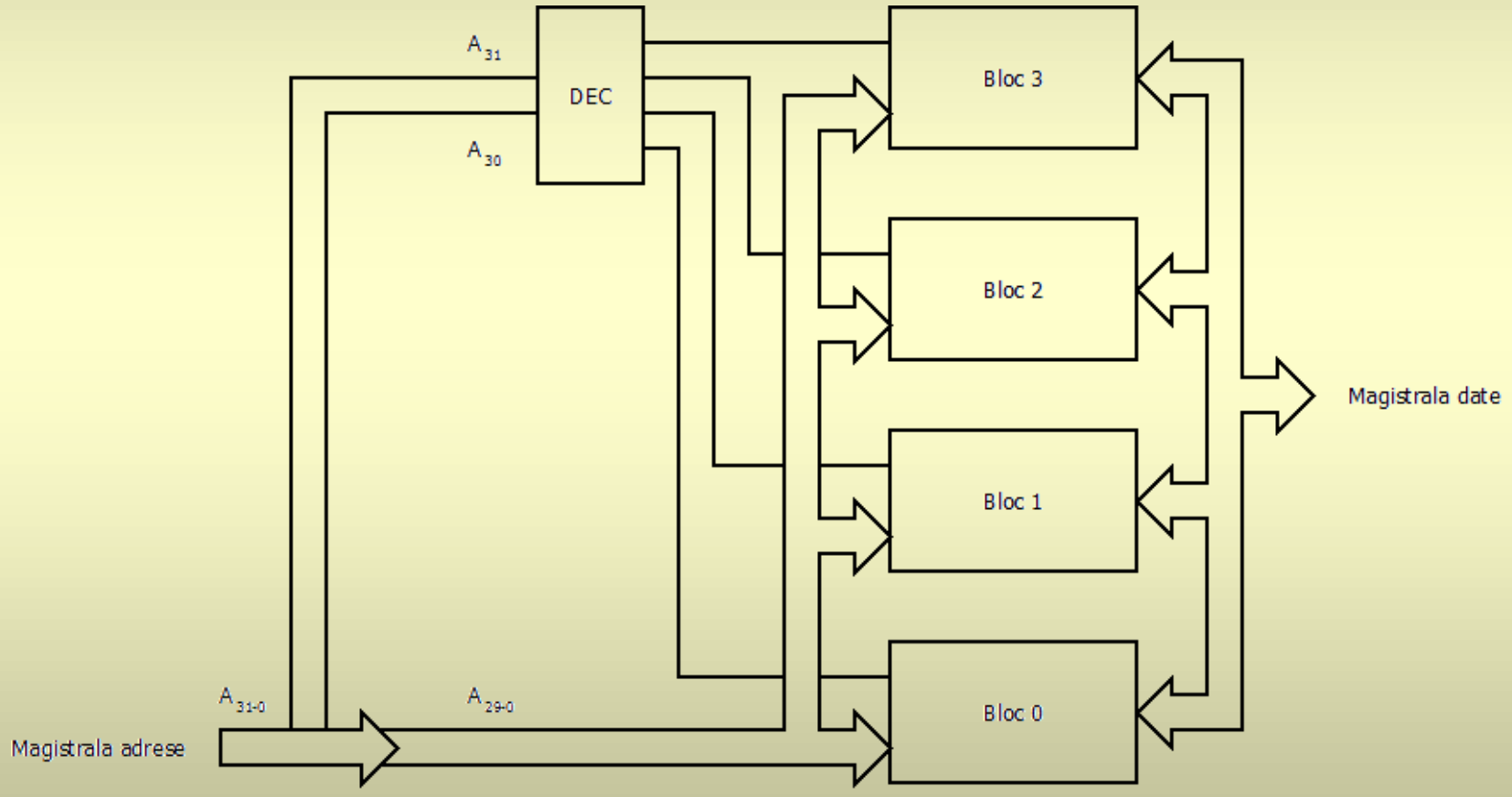
Spațiul adreselor de memorie

- limitat de capacitatea magistralei de adrese
  - deci este un element al arhitecturii calculatorului
  - exemplu: procesoare pe 32 biți - cel mult 4 GB ($=2^{32}$)
- nu se poate conecta memorie cu o capacitate mai mare decât maximul permis de magistrala de adrese
  - dar se poate mai puțin

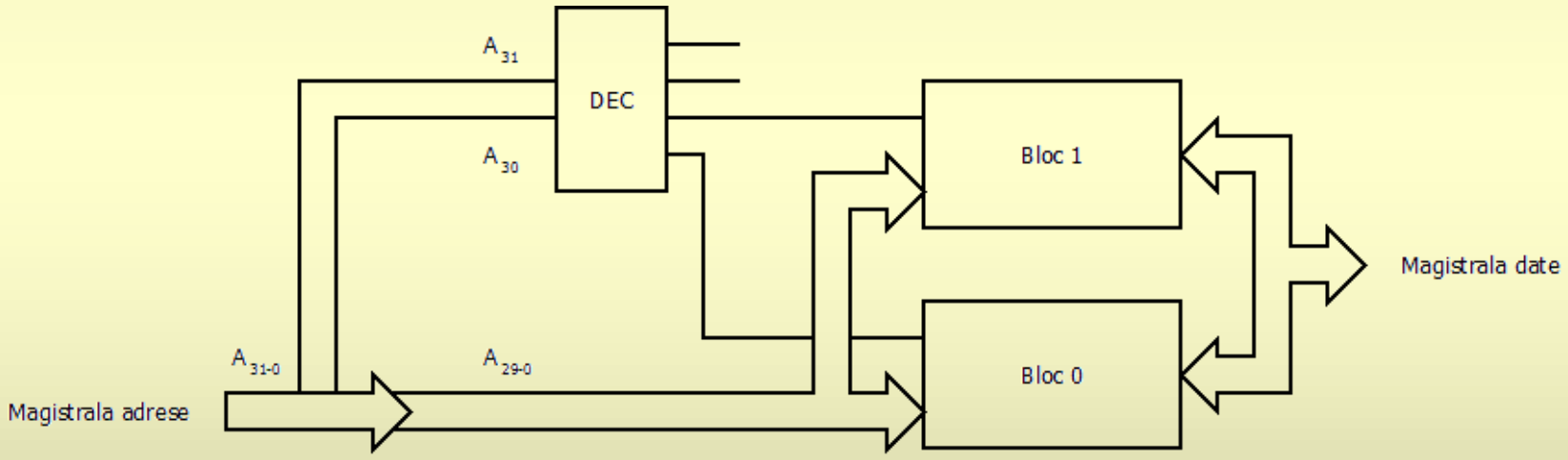
Decodificarea adreselor

- exemplu: procesor pe 32 biţi
- avem 4 circuite (blocuri) de memorie de câte 1 GB fiecare
  - total 4 GB - maximum posibil
- pentru o adresă dată, cum este selectat circuitul potrivit de memorie?
- dar dacă avem doar 2 circuite de câte 1 GB?

Decodificare totală



Decodificare parţială



Tipuri de adrese

- absolută
  - – numărul de ordine asociat unei celule
  - – numerotarea începe de la 0
- relativă
  - – poziţia faţă de un octet de referinţă
  - – exemplu: indicele unui element într-un tablou
- simbolică
  - – identificator alfanumeric ataşat unei adrese
  - – exemplu: numele unei variabile

Ordinea octeţilor (1)

- o variabilă poate ocupa mai multe locații de memorie
  - consecutive
- exemplu: variabilă de tipul *unsigned int*
  - ocupă 4 octeți
  - presupunem că adresele ocupate sunt 150÷153
  - care dintre octeți se găsește la adresa cea mai mică? dar la cea mai mare?

Ordinea octeţilor (2)

- decizia este luată la proiectarea unității de procesare (CPU)
  - care realizează citirile și scrierile în memorie
- variante
  - *little endian* - octetul cel mai puțin semnificativ la adresa cea mai mică
  - *big endian* - octetul cel mai semnificativ la adresa cea mai mică

Exemplu

unsigned int x = 0xB67A49E3; // $B67A49E3_{(16)}$

little endian

adresă	valoare
	. . .
153	$B6_{(16)} = 10110110_{(2)}$
152	$7A_{(16)} = 01111010_{(2)}$
151	$49_{(16)} = 01001001_{(2)}$
150	$E3_{(16)} = 11100011_{(2)}$
	. . .

big endian

adresă	valoare
	. . .
153	$E3_{(16)} = 11100011_{(2)}$
152	$49_{(16)} = 01001001_{(2)}$
151	$7A_{(16)} = 01111010_{(2)}$
150	$B6_{(16)} = 10110110_{(2)}$
	. . .

IV.3. Memoria cache

Problema

- procesorul este mai rapid decât memoria
  - este obligat să aştepte până când primeşte datele şi instrucţiunile din memorie
- performanţa procesorului nu este exploatată
- cauze
  - dezvoltarea tehnologică
  - factori economici

Soluţia

- legile localizării - determinate empiric
- valabile pentru marea majoritate a aplicaţiilor
- tipuri de localizare
  - spaţială
  - temporală
- exploatarea localizării - ierarhia de memorii

Localizare temporală

- dacă o locaţie de memorie este accesată la un moment dat, este foarte probabil să fie accesată din nou în viitorul apropiat
- exemple
  - variabilele sunt folosite în mod repetat
  - bucle de program - instrucţiunile se repetă

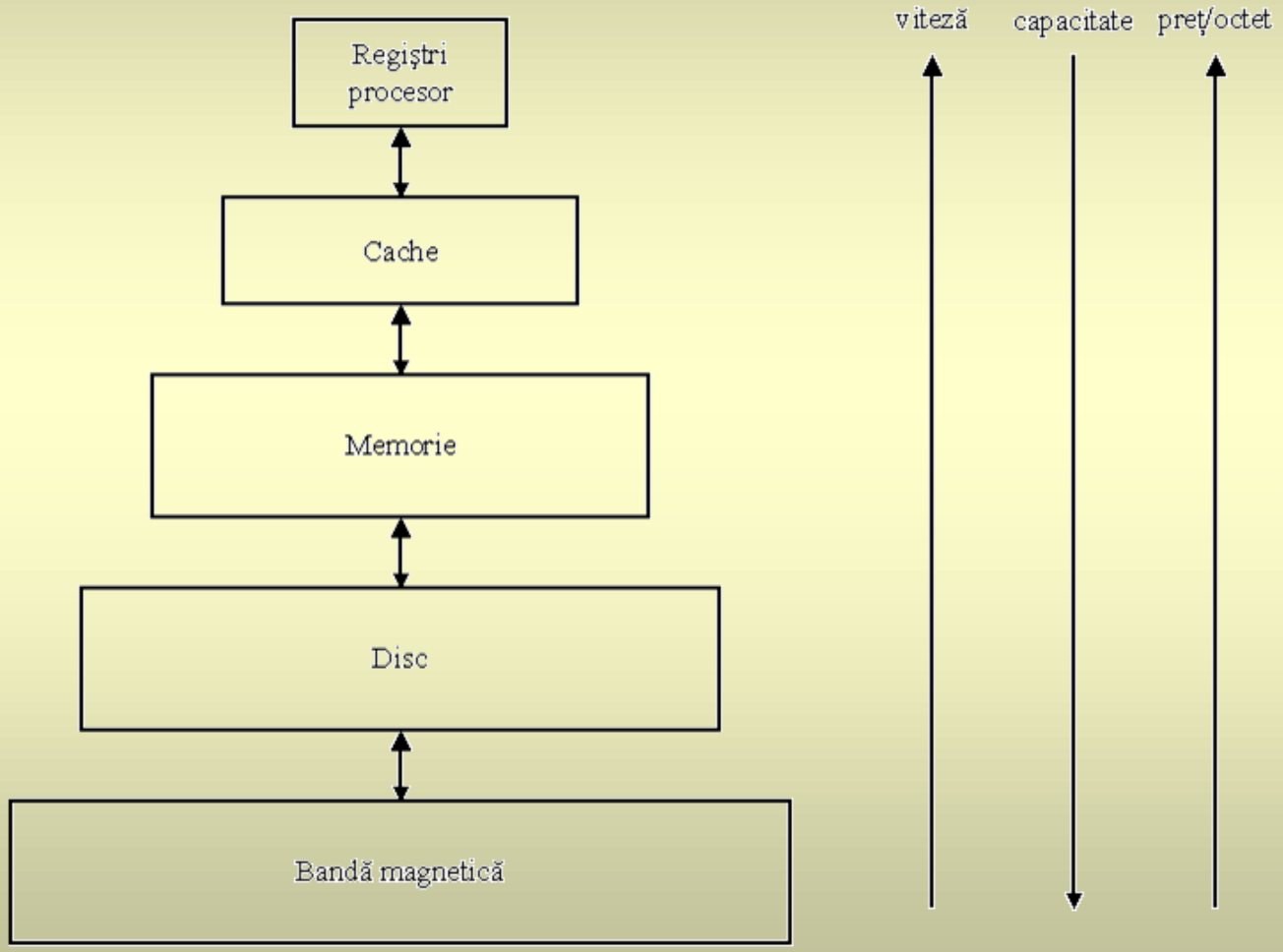
Localizare spaţială

- dacă o locaţie de memorie este accesată la un moment dat, este foarte probabil ca şi locaţiile vecine să fie accesate în viitorul apropiat
- exemple
  - parcurgerea tablourilor
  - execuţia secvenţelor de instrucţiuni - aflate la adrese consecutive

Ierarhia de memorii (1)

- diferite nivele de stocare
  - având caracteristici diferite
- cerințe contradictorii
  - capacitate cât mai mare
  - viteză de acces cât mai mare
- aceste cerințe nu pot fi satisfăcute simultan de același tip de stocare

Ierarhia de memorii (2)



Cum funcționează?

- toate informațiile sunt memorate pe nivelul cel mai de jos
- pe nivelele superioare sunt aduse subseturi tot mai mici din aceste informații
- problema - cum maximizăm viteza?
- reformulare: care subseturi trebuie aduse pe nivelele superioare?
  - pentru o viteză cât mai mare

Legile localizării - consecinţe

- la un moment dat, un program foloseşte numai o (mică) parte din locaţiile sale de memorie
  - acestea trebuie păstrate pe nivelele superioare
  - mai mici, dar mai rapide
- care sunt aceste locații?
  - cel mai probabil, cele accesate cel mai recent
- caz particular - memoria cache

Memoria cache

- circuit rapid şi de capacitate mică
  - interpus între procesor şi memoria principală
- reţine locaţiile din memoria principală accesate cel mai recent de procesor
- procesorul solicită conţinutul unei locaţii
  - se caută mai întâi în cache
  - dacă locaţia nu este găsită în cache - căutare în memoria principală

Caracteristici

Viteza

- foarte mare - la acelaşi nivel cu procesorul
  - tehnologie performantă (SRAM)
  - dimensiune redusă → bloc de decodificare mai simplu, deci mai rapid

Preţul

- rezonabil
  - datorită dimensiunii reduse

Parametri de performanţă (1)

- H - rata de succes (*hit ratio*)
  - procentajul cazurilor în care locaţia căutată a fost găsită în cache
- M - rata de insucces (*miss ratio*)
  - procentajul cazurilor în care locaţia căutată nu a fost găsită în cache

$$0 \leq H, M \leq 1$$

$$M = 1 - H$$

Parametri de performanţă (2)

- T_c - timpul de acces la cache
- T_m - timpul de acces la memorie în cazul unei ratări în cache
- T - timpul mediu de acces la memorie (cu cache)
- T_p - timpul de acces la memoria principală (în absenţa cache-ului)

Performanţa memoriei cache (1)

$T = T_c \cdot H + T_m \cdot M$

- dacă $T < T_p \rightarrow$ spor de viteză
- T - mărime statistică
- cazuri extreme
  - H=100% (M=0): $T = T_c \rightarrow$ ideal
  - H=0 (M=100%): $T = T_m \rightarrow$ pierdere de viteză ($T_m > T_p$)

Performanţa memoriei cache (2)

Situaţia reală - exemplu

- $T_c = 2$ ns
- $T_p = 10$ ns
- $T_m = 11$ ns
- $H = 95\%$
- $T = 2.45$ ns $= 0.245\ T_p \rightarrow$ viteza de acces creşte de peste 4 ori

Adresare

- adresa din cache nu corespunde cu adresa din memoria principală
- căutarea se face după adresa din memoria principală
- deci cache-ul trebuie să reţină şi adresele locaţiilor în memoria principală

Linii de cache

- cache-ul se foloseşte de localizarea temporală
- cum se poate exploata şi localizarea spaţială?
- când se aduce o locaţie în cache, se aduc şi locaţiile vecine
  - acestea formează o *linie de cache*

Politica de înlocuire

- cache mic - se umple repede
- noi linii aduse în cache - trebuie eliminate altele mai vechi
  - eliminare - scriere în memoria principală
- care linii trebuie eliminate?
  - scopul - creşterea vitezei
- cele care nu vor fi accesate în viitorul apropiat !!!

Îmbunătăţirea performanţei

- depinde de două mărimi
  - timpul de acces la cache (T_c)
  - rata de succes (H)
- nu pot fi optimizate simultan
- influenţate de
  - tehnologie
  - politica de înlocuire

Tipuri constructive de cache

- cu adresare directă (*direct mapped cache*)
- total asociativ (*fully associative cache*)
- parţial asociativ (*set associative cache*)